Lab5 - Coverage

In a digital design verification process coverage defines how thoroughly the design has been exercised by the testbench, quantifying which parts of the code, functionality, and scenarios have been observed during simulation.

Coverage in hardware verification is typically divided into two complementary categories:

- Code coverage measures how much of the design's implementation has been executed during simulation — for example, which statements, branches, and finite state machine states were reached.
- Functional coverage, on the other hand, is defined by the verification engineer and tracks whether the intended behaviors, corner cases, and protocol scenarios have been exercised.

Together, these metrics provide a quantitative view of verification progress: code coverage ensures that the testbench is not leaving parts of the RTL untouched, while functional coverage ensures that the design's specification has been thoroughly validated. High coverage is not a guarantee of correctness, but low coverage is a clear indicator of insufficient testing. In this lab, we will practice both code and functional coverage, learning how to interpret reports and how to enhance stimulus to close coverage gaps.

1. Get Ready for this Lab

In the web of the course at https://docencia.ac.upc.edu/master/MIRI/PV/labs.html, for session Lab5, you will find two packages fifo.tgz and cache.tgz containing two directories with the following files:

a) fifo.tgz: Makefile, fifo.sv, and tb_fifo.sv.
b) cache.tgz: Makefile, simple_cache.sv, and tb_simple_cache.sv

The FIFO design will be used to learn how to execute and gather coverage data with QuestaSim. After that, you will apply what you've learned by adding coverage to a basic cache design on your own.

2. How Coverage Works in QuestaSim

To demonstrate the process of generating coverage reports in QuestaSim, we will utilize the provided FIFO example. Open the design (fifo.sv) and the simple testbench (tb_fifo.sv). The design consists of a simple circular FIFO buffer (as the one discussed in the UVM intro lecture), where we can write (push) and read (pop) data, though we cannot do both operations at the same time (in such case, no action is done). It also generates full/empty flags to report the state of the buffer.

On the other hand, the provided testbench simply generates few random tests to stimulate the design. It includes two immediate assertions to test writing to a full or reading from an empty FIFO in simulations. Note that these assertions are merely informative in the testbench, no abort is produced. Also, there is a covergroup section (activated on posedge clk) to measure functional coverage for different situations/combinations.

As in previous labs, you can run QuestaSim from command line (commonly using a Makefile) or from GUI environment. Let's discuss both.

2.1. Command Line

To enable coverage, follow these steps whether you are using the command line or the GUI:

a) Code coverage: this type of coverage is automatically generated by the tool (you do not need to add anything to your testbench). You can enable it at compile time adding "+cover" to your vlog commands. That will generate complete code coverage for your run. However, you can produce specific code coverage with the following options:

Argument	Type of code coverage
`+cover=s`	Statement coverage
`+cover=b`	Branch coverage
`+cover=c`	Conditional coverage
`+cover=e`	Expression coverage
`+cover=f`	FSM coverage
`+cover=t`	Toggle coverage
`+cover=x`	Extended toggle coverage

- It is also possible to combine several of these options to create multiple types of coverage. e.g., `+cover=sce` (to generate statement, conditional, and expression coverage)

b) Functional coverage: this type of coverage will come from the covergroup/coverpoint statements you added to your testbench. To generate data for them to need to set option "-coverage" to your vsim command.

When running simulations from command line the most common methodology is to dump the coverage data into a file and generate a report later out of this data. Have a look at the Makefile provided. In the "run" target, you can observe the parameters specified under the "-do" option.

```
-do "coverage save -onexit test1.ucb; run -all; quit -f"
```

This command saves coverage data to test1.udb and runs the test. Then, the most basic way to report your coverage is under the "report" target in the Makefile, using the vcover command:

```
vcover report test1.ucb
```

This will generate a simple report showing code and functional coverage percentages. You can add "-details" to that line to show a much more detailed output.

If you prefer, you can also generate the coverage report in HTML format. For that, look at the "report_html" target in the Makefile:

```
vcover report -html -output coverage_report test1.ucb
```

Try it out and have a look at `coverage_report/index.html` in your browser.

Lastly, as we discussed in class, you can combine multiple coverage reports from different .ucb files if you've run several tests and want a combined report by using "vcover merge".

`vcover` is a quite powerful command. You can do "`vcover -help`" or "`vcover report -help`" to get more options.

2.2. GUI

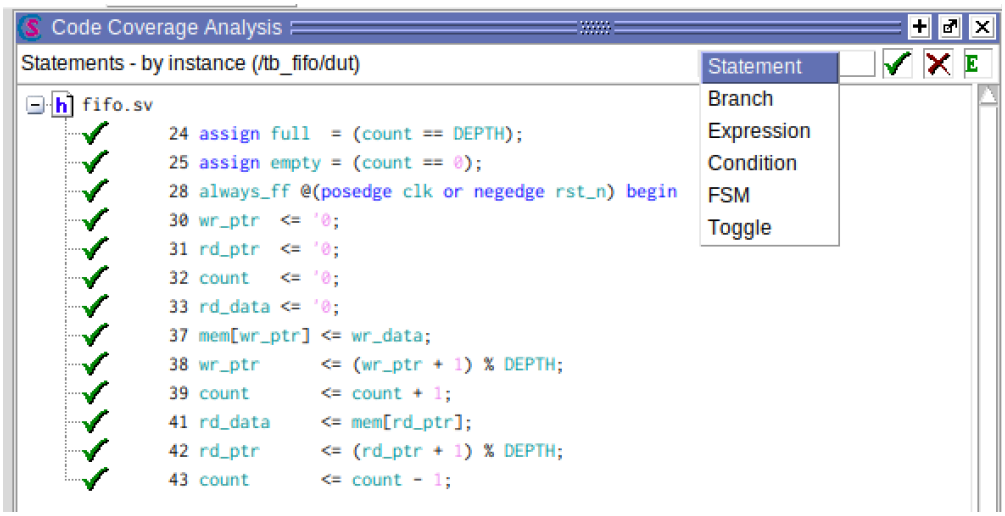
Once you have opened vsim in GUI mode, created a workspace and compiled your sources with "+cover" option, you can invoke the vsim command as:

```
vsim -coverage work.tb_fifo
```

You will see a new set of windows is opening:

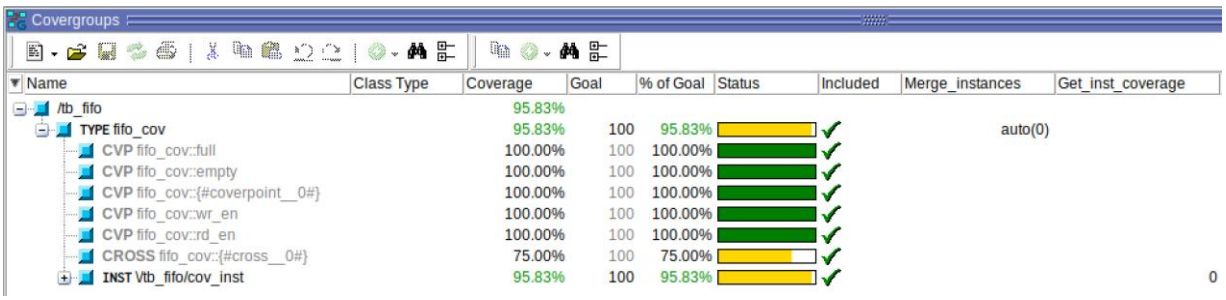
- Analysis
- Assertions
- Cover Directives
- Covergroups

You can now simulate with "run -all". From the "Analysis" tab now you can pick what to show (Statement/ Branch/Expression/Condition/FSM/Toggle) for your code coverage (note that this is by instance: in this simple example you can pick /tb_fifo or /tb_fifo/dut):



You can also open the source code from tb_fifo.sv and fifo.sv and find embedded coverage data.

Finally, in the "Covergroups" tab you can have a look at your functional coverage status:



3. Design: FIFO

Look at the coverage section of the testbench. Learn from it. You can see we added a covergroup and a cover property (Yes! Like for assertions! Though here the meaning is "our test exercised this property")

HINT: Play with detailed command-line report or GUI

Task #1

The testbench, as provided, has the stimulus commented out with /* */. Execute as is and report coverage. Just doing "make report" it should show at the end something like this:

```
TOTAL COVERGROUP COVERAGE: 43.05%  COVERGROUP TYPES: 1
```

First thing to notice: we did basically nothing at the coverage tool already reports 43.05%

Question #1.1: The output says we have 13 covergroup bins + 1 directive. Which ones are?
Question #1.2: Which ones out of these 13 + 1 have been covered just with the reset sequence?

Task #2:

Enable the test provided (just remove the /* */) and repeat the experiment.

Question #2.1: Which ones of the 13 covergroup bins + 1 cover property have now been covered?

Task #3:

Add additional tests to enhance our testbench and reach a 100% covergroup bins + properties covered. You can put them in the testbench code where it says "ADD ADDITIONAL STIMULUS AS NEEDED HERE"

4. Design: Simple Cache

It is your turn to include cover groups or properties in another design. Continuing our fascination with CPU pipeline components, today we are focusing on cache memory.

4.1. Specification

Overview

The simple_cache module implements a direct-mapped cache with parameterizable line count, line size, address width, and data width. It supports read and write operations, tracks valid bits and tags, and reports whether an access is a hit or a miss. On a miss, the cache simulates fetching data from memory by returning the address value.

Parameters

Parameter	Description	Default
`CACHE_LINES`	Number of cache lines (sets)	16
`LINE_SIZE`	Number of words per line (block size)	4
`ADDR_WIDTH`	Width of the address bus in bits	8
`DATA_WIDTH`	Width of each data word in bits	32

Ports

Signal	Direction	Width	Description
`clk`	Input	1	Clock signal
`reset`	Input	1	Asynchronous reset (active high)
`read`	Input	1	Read enable
`erite`	Input	1	Write enable
`addr`	Input	ADDR_WIDTH	Address of the access
`data_in`	Input	DATA_WIDTH	Data to be written
`data_out`	Output	DATA_DIWTH	Data returned on read
`hit`	Output	1	Indicated whether access was a cache hit

Internal Structures

- Cache memory: cache_mem[CACHE_LINES][LINE_SIZE] stores data words.
- Tag array: tag_array[CACHE_LINES] stores the tag for each line.
- Valid array: valid_array[CACHE_LINES] indicates whether a line contains valid data.

Address Decoding

The address is split into three fields:
- Tag: addr[ADDR_WIDTH-1 -: TAG_WIDTH]
- Index: addr[OFFSET_WIDTH+INDEX_WIDTH-1 -: INDEX_WIDTH]
- Offset: addr[OFFSET_WIDTH-1 -: OFFSET_WIDTH]

Behavior

- Reset
  - All valid bits cleared (valid_array[i] = 0).
  - All tags and cache memory words set to zero.
  - Outputs (hit, data_out) cleared.
- Read Operation
  - If valid_array[index] == 1 and tag_array[index] == tag:
    - Hit: hit = 1, data_out = cache_mem[index][offset].
  - Else:
    - Miss: hit = 0.
    - Simulated memory fetch: data_out = {24'h0, addr}.
    - Cache line updated: cache_mem[index][offset] = {24'h0, addr}, tag_array[index] = tag, valid_array[index] = 1.
- Write Operation
  - If valid_array[index] == 1 and tag_array[index] == tag:
    - Hit: hit = 1.
  - Else:
    - Miss: hit = 0.
    - Tag and valid bit updated: tag_array[index] = tag, valid_array[index] = 1.
  - In both cases: cache_mem[index][offset] = data_in.
- No Operation
  - If neither read nor write is asserted, outputs remain stable (hit = 0).

Limitations/Simplifications

- Read operations take precedence over write operations: If both are requested simultaneously, only the read operation will be executed.
- No replacement policy: Direct-mapped cache always overwrites the line at index.
- No write-back or write-through policy: Writes update cache only; no backing memory modeled.
- Miss handling: Simplified — returns {24'h0, addr} instead of fetching from real memory.
- No stall or wait states: All operations complete in one cycle.

4.2. Testbench

You will find a provided testbench and Makefile. You can use it as basis for your development.

Task #4:

Add cover groups/cover properties to your testbench and report the status with the provided random test.

Task #5:

Enhance your tests to accomplish 100% coverage (not required to toggle)

5. What to Turn In

Please provide your testbench and a single PDF document with the following:

1. Please indicate how many hours you spent on this lab. This will not affect your grade, but will be helpful for calibrating the workload for the future.
2. Document your findings for each assigned task within the FIFO module:
    a. (1 pt) Task 1: Question 1.1
    b. (1 pt) Task 2: Question 1.2
    c. (1 pt) Task 2: Question 2.1
    d. (2 pts) Task 3
3. Document your findings for each assigned task within the cache module:
    a. (2 pts) Task 4
    b. (3 pts) Task 5